Project 2 – Acme Analytics Linux Platform

Day 11 · Phase 4: memory limit + OOM killer · disk, RAM, boot, mounts, live vs permanent

Muhammad Bilal Javed · 3 November 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01 (memory-limit test, clean-up check)
Status	Phase 4 🟡 – Steps 4.1–4.4 ✅ · Step 4.5: concept ✅ · lab Step 1 ✅ · **lab Step 2 ✅ memory limit proved with the OOM killer** · foundations ✅ (boot, page cache, mounts, live vs permanent) – Steps 3–7 next
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot or diagram. Terms are explained in the glossary. Previous day: Day 10.

---

1. Lab Step 2 (app01) – a hard memory limit and the OOM killer

💡 **In plain words:** I gave a test program a box with a 100 MB memory wall and no swap, then asked it to use 300 MB. The kernel stopped it at the wall – inside its own box only. app01 and every service next to it kept running.

🔧 **Technical:**

```
sudo systemd-run --scope -p MemoryMax=100M -p MemorySwapMax=0 stress-ng --vm 1 --vm-bytes 300M --timeout 1s
sudo journalctl -k --since "5 min ago" | grep -i -E "oom|killed" | tail -5
```

The pattern behind every test:

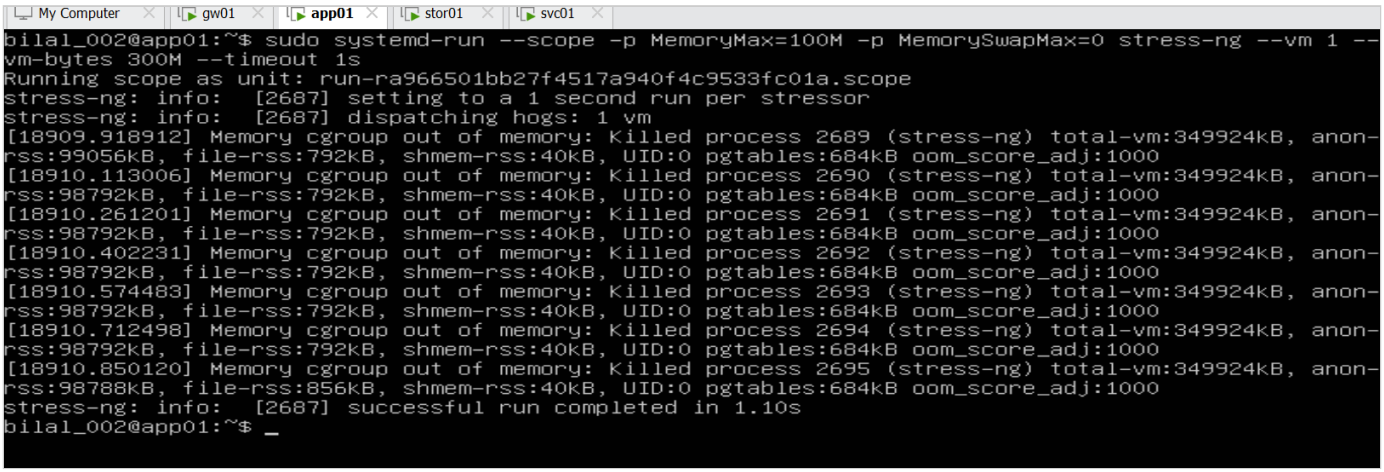
```
sudo systemd-run --scope  -p <LIMIT>   stress-ng <LOAD>  --timeout <TIME>
     └─ make a box ──┘  └ the limit ┘  └── fake load to test the limit ──┘
```

Part	Meaning
`systemd-run --scope`	a temporary box for my command – output on my screen, the box is deleted when the command ends
`-p MemoryMax=100M` · `-p MemorySwapMax=0`	hard memory wall of 100 MB · no escape to swap
`stress-ng --vm 1 --vm-bytes 300M`	1 memory worker that tries to use 300 MB (`--cpu` = CPU load, `--fork` = new processes)
where the box goes	with sudo the request goes to the system systemd (PID 1) → default place **system.slice**

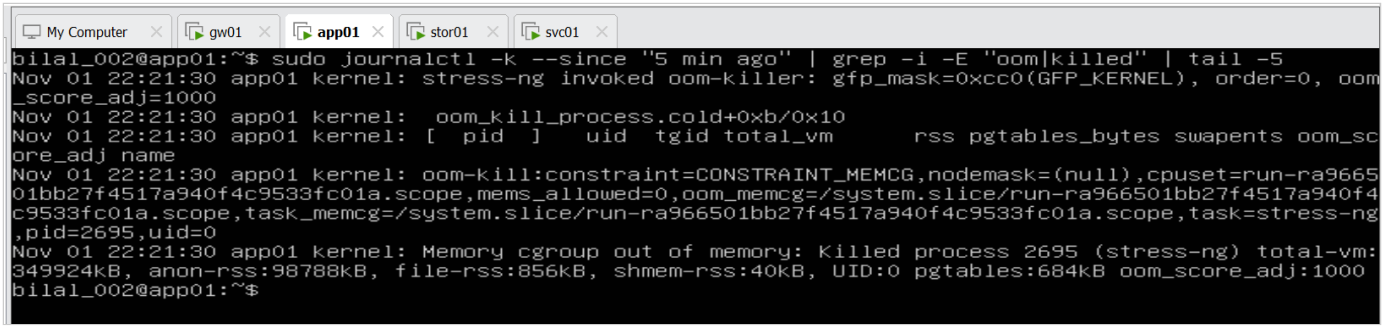
What happened:

Line	Meaning
`Running scope as unit: run-ra9665….scope`	systemd made the temporary box
`total-vm:349924kB`	the worker wanted about 340 MB
`anon-rss:98792kB`	it really got about 96 MB of RAM – right at the 100 MB wall
`Killed process 2689 … 2695` (7 times)	the OOM killer stopped the worker; stress-ng restarted it, each time with a new PID
`[2687] successful run completed`	the parent was never killed – only its workers
`constraint=CONSTRAINT_MEMCG`	killed because of the **box limit**, not because app01 ran out of RAM
`oom_memcg=/system.slice/run-ra9665….scope`	the box that hit the limit – only this box was touched

📸 **Evidence:**



OOM test – stress-ng killed at 100 MB



Journal proof – CONSTRAINT_MEMCG

Clean-up check before the next test: `pgrep -a stress-ng` (nothing = no test running) · `systemctl list-units --all | grep -E "run-|mytest"` – the `run-….mount` lines are normal mounts under `/run`, not old tests; old test boxes would end in `.scope` or be called `mytest.service`. Leftovers: `sudo pkill stress-ng` · `sudo systemctl stop mytest` · `sudo systemctl reset-failed`.

Same idea in containers: a Kubernetes pod shows `OOMKilled`, Docker kills a container over `--memory`.

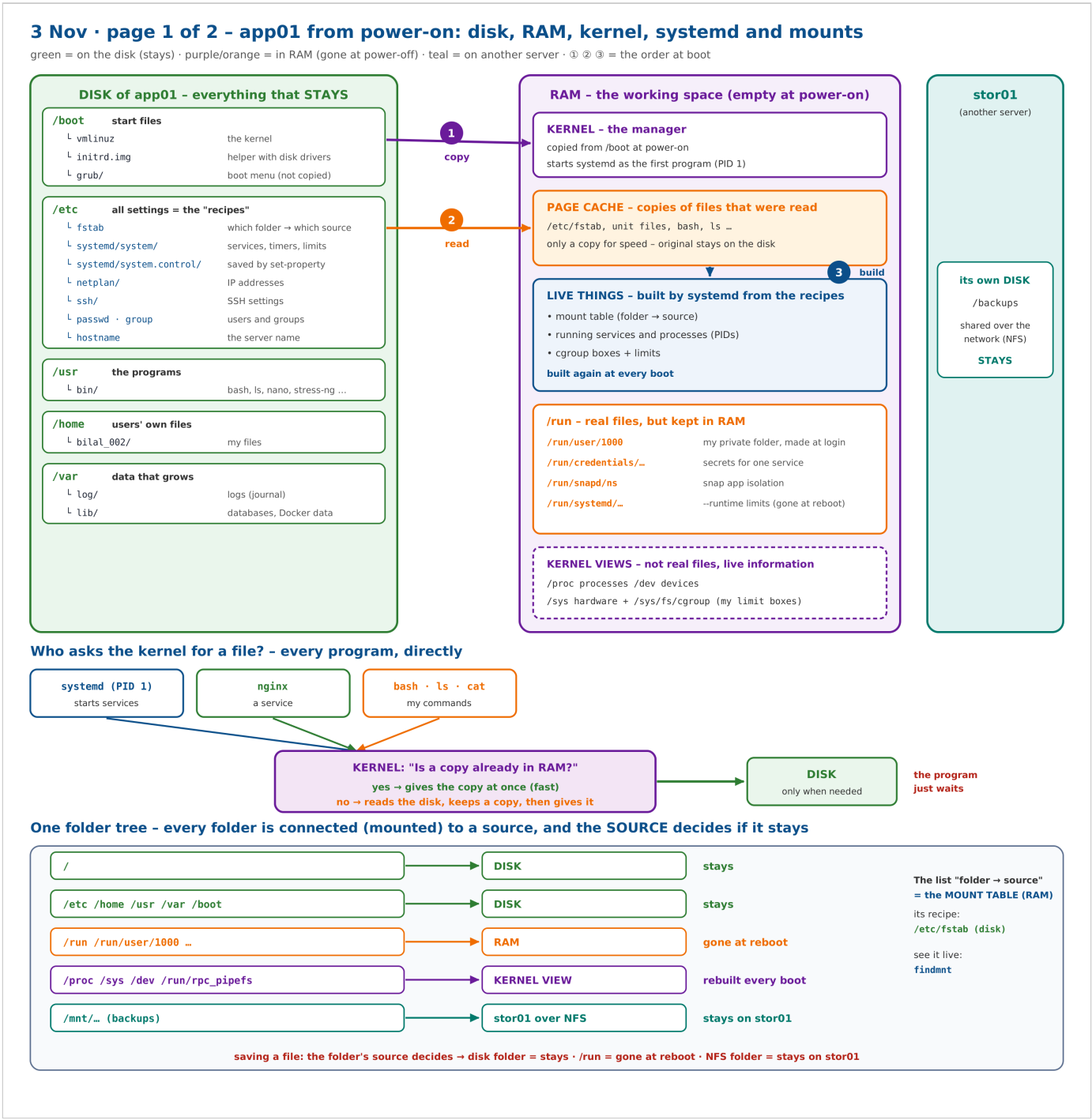
---

2. How Linux uses the disk and RAM – from power-on

💡 **In plain words:** everything is stored on the disk; RAM starts empty. The kernel is copied into RAM and becomes the manager, systemd reads its settings from `/etc`, and builds the live things in RAM.

1. ① **Copy:** two files from `/boot` are copied into RAM – `vmlinuz` (the kernel) and `initrd.img` (a small helper with disk drivers). The rest of `/boot` and all other files stay on the disk.
2. The kernel starts **systemd** as the first program (PID 1).
3. ② **Read:** systemd reads its recipes in `/etc` (fstab, unit files, limits). Reading = the kernel puts a **copy of that one file** in RAM (the page cache); the original stays on the disk.
4. ③ **Build:** from the recipes systemd builds the live things in RAM – the mount table, the running services and the cgroup boxes. They are built again at every boot.

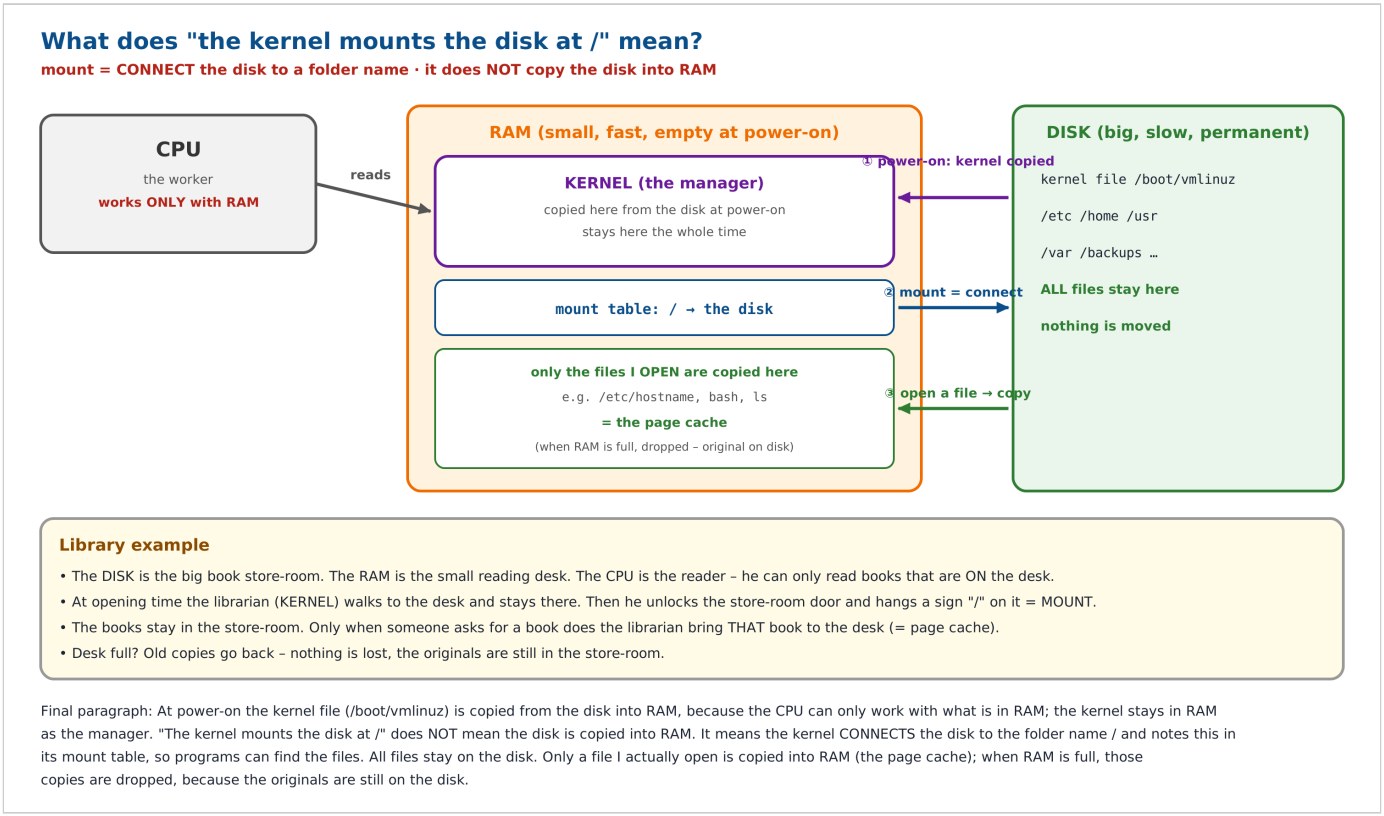
📸 **Diagram – the whole picture on one page:**



app01 from power-on – disk, RAM, kernel, systemd and mounts

2.1 Mount = connect, not copy

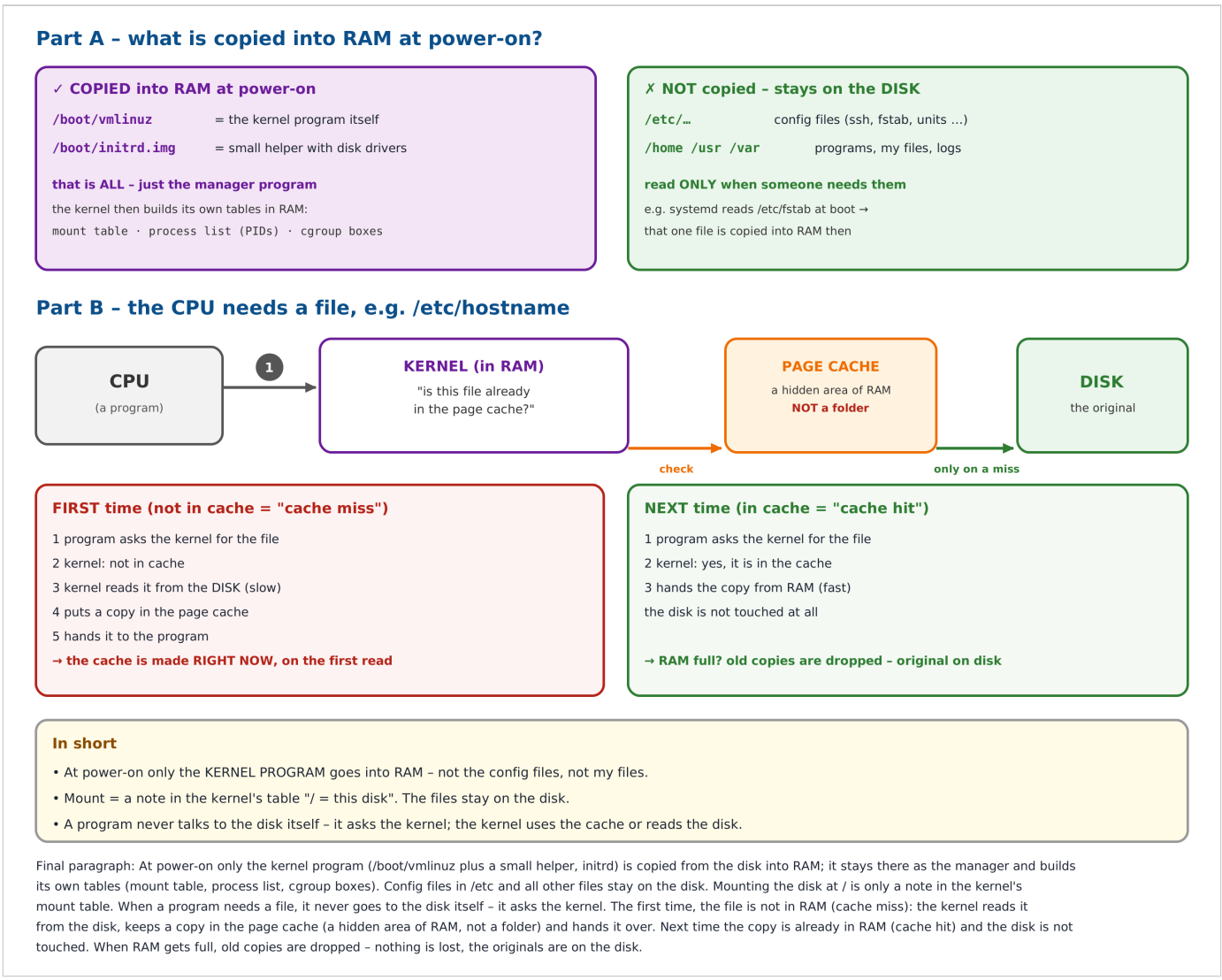
A mount connects a **folder** to a **source**. The files stay where they are; the kernel only notes "/ = this disk" in its mount table (in RAM).



What mount means – the library example

2.2 When a program needs a file

Every program – systemd, nginx, or my own `ls` – asks the **kernel**. Copy already in RAM → given at once (fast). Not in RAM → the kernel reads the disk, keeps a copy, then gives it. The program simply waits. The page cache is a hidden area of RAM, not a folder.



How a file is read – cache miss and cache hit

2.3 One folder tree – the source decides if it stays

Folders	Source	After reboot
`/ /etc /home /usr /var /boot`	disk	stays
`/run` (e.g. `/run/user/1000`)	RAM (tmpfs)	gone
`/proc /sys /dev /run/rpc_pipefs`	kernel view – live information, not real files	rebuilt
`/mnt/…` backups	stor01 over NFS	stays on stor01

systemd tracks every mount as a `.mount` unit named after its folder with `/` → `-` (a real dash = `\x2d`), e.g. `run-user-1000.mount` = `/run/user/1000`. Check: `findmnt`.

---

3. Live test vs permanent – where I write it decides

💡 **In plain words:** the work always happens in RAM. Saving a file in a disk folder is **not** lost – the kernel writes it to the disk within seconds. What is lost at reboot is a **live change with no file**. `/etc` is special because of its job: it holds the settings programs read every time they start.

The change	Live – works at once, gone at reboot	File on the disk – stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1` (= `/proc/sys/…`)	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service` + `enable`
a limit	`systemd-run -p … · set-property --runtime`	`set-property` · `…service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`

Admin habit: try it live → check it works → write it into the file so it stays. A cgroup box with no recipe file comes back after reboot with the default settings and no limit – that is why my `systemd-run` tests left nothing behind.

📸 **Diagrams:**

Two kinds of change – which one survives a reboot?

saving a file in a disk folder is NOT lost · only changes that live ONLY in RAM are lost

KIND 1 – I SAVE A FILE in a disk folder → STAYS

e.g. nano /home/bilal_002/notes.txt · edit /etc/fstab · a new unit file in /etc/systemd/system/

I press save
(in nano, cp, echo >)

→

copy in RAM
(page cache, updated)

→

kernel writes it to the DISK
automatically, within a few seconds

→ the file is ON THE DISK. After logout or reboot it is still there. The RAM copy was only for speed.

(only a sudden power cut in those few seconds could lose it – a normal shutdown writes everything first)

KIND 2 – a LIVE CHANGE with no file → LOST at reboot

the command changes only the kernel's live tables in RAM – nothing is written to any file on the disk

`sudo renice -n 10 -p 1234`	priority of one process
`sudo systemd-run --scope -p MemoryMax=100M …`	temporary box + limit
`sudo systemctl set-property --runtime …`	limit kept in /run (RAM)
`sudo ip addr add 192.168.60.99/24 dev ens33`	extra IP address
`sudo mount 192.168.60.20:/backups /mnt`	a mount typed by hand

To make it PERMANENT:

write it into a RECIPE FILE

- limit → unit file / drop-in
or set-property (no --runtime)
- Nice= → in the unit file
- IP → /etc/netplan/
- mount → /etc/fstab

→ next boot: systemd rebuilds everything from the recipes only. A box with no recipe comes back with the DEFAULT settings (no limit).

This is why my systemd-run tests left nothing behind – they were live changes only.

My wording – corrected

- Not all green (disk) things come into RAM – only the files a program actually needs, when it needs them.
- Working "in RAM" does NOT mean my saved work is lost: a file saved in /home, /etc or /usr is written to the disk.
- /usr holds the programs (bash, ls). I run them from there; my own files go in /home.
- What is lost at reboot: changes that exist ONLY in RAM – live commands without a file, and everything in /run.
- cgroups: the boxes are rebuilt every boot from the recipe files – no recipe = default box with no limit. Correct!
- Mount = connect a folder to a source (disk, RAM, another server). Saving into that folder puts the file on that source.

Final paragraph: The kernel is copied into RAM and starts systemd. Whenever systemd or any other program needs a file, the kernel copies only that file from the disk into RAM (page cache); the original stays on the disk. There are two kinds of change. KIND 1 – saving a file in a disk folder (/home, /etc …): the kernel writes it to the disk within seconds, so it is still there after logout or reboot. KIND 2 – a live change with no file (renice, systemd-run, set-property --runtime, ip addr add, a hand-typed mount): it lives only in the kernel's tables in RAM and is lost at reboot. To make a live change permanent, write it into a recipe file on the disk (unit file or drop-in, /etc/netplan, /etc/fstab). At the next boot systemd rebuilds everything – including the cgroup boxes – only from those recipes; a box without a recipe comes back with the default settings and no limit.

Two kinds of change

The work always happens in RAM – WHERE you write it decides if it stays

same change, two places: the LIVE way (try it now, gone at reboot) or the FILE in /etc (stays)

the change	LIVE – in RAM / kernel works at once · gone at reboot	FILE in /etc – on the disk read at every boot · stays
IP forwarding on gw01 (Phase 1)	`sudo sysctl -w net.ipv4.ip_forward=1` = writes into /proc/sys/net/ipv4/ip_forward	`/etc/sysctl.conf or /etc/sysctl.d/…conf` net.ipv4.ip_forward = 1
a service run a program as a service	`sudo systemd-run --unit=mytest …` temporary service, gone at reboot	`/etc/systemd/system/mytest.service` + systemctl enable → starts every boot
a limit CPU / memory of a box	`systemd-run -p MemoryMax=100M …` set-property --runtime (→ /run)	`set-property (no --runtime) or` …/xyz.service.d/limits.conf
a mount connect a folder to a source	`sudo mount 192.168.60.20:/backups /mnt` typed by hand	`/etc/fstab` one line per mount
an IP address on a network card	`sudo ip addr add 192.168.60.99/24 dev ens33` typed by hand	`/etc/netplan/…yaml` + sudo netplan apply

/proc /sys /dev /run – what are they for?

- to SEE the system live: processes (/proc), hardware and cgroup boxes (/sys), devices (/dev), running state (/run)
- to CHANGE something live – it works at once but is gone at reboot → perfect to TRY a setting first
- **admin habit: try it LIVE → check it works → then write it into the FILE in /etc so it stays**

Final paragraph: The work always happens in RAM, and what stays depends on where I write it. Most settings can be changed in two ways: LIVE, through a command that changes the kernel's tables or the folders /proc, /sys and /run – it works at once but is gone at reboot – or in a FILE in /etc on the disk, which is read at every boot and therefore stays. IP forwarding on gw01 is the perfect example: sysctl -w (or /proc/sys/net/ipv4/ip_forward) is live, /etc/sysctl.conf is permanent. The same pattern fits services (systemd-run vs a unit file in /etc/systemd/system plus enable), limits (--runtime vs a drop-in), mounts (mount command vs /etc/fstab) and IP addresses (ip addr vs /etc/netplan). /proc, /sys, /dev and /run are for seeing and changing the system live, which makes them good for trying a setting first; the admin habit is: try it live, check it works, then write it into the file so it stays.

Live vs file

3 Nov · page 2 of 2 – live test vs permanent, and my memory-limit test on app01

the big idea of the day: the work always happens in RAM – WHERE you write it decides if it stays

A · Every setting has two ways: LIVE (a test, gone at reboot) or a FILE in /etc (stays)

the change	LIVE – works at once · gone at reboot	FILE on the disk – read at every boot · stays
IP forwarding (gw01)	`sysctl -w net.ipv4.ip_forward=1 (/proc/sys/…)`	`/etc/sysctl.conf`
a service	`systemd-run --unit=mytest …`	`/etc/systemd/system/mytest.service + enable`
a limit	`systemd-run -p MemoryMax=… · set-property --runtime`	`set-property · …service.d/limits.conf`
a mount	`mount 192.168.60.20:/backups /mnt`	`/etc/fstab`
an IP address	`ip addr add …`	`/etc/netplan/…yaml`
a saved file	– (saving in /home, /etc … already goes to disk)	any folder on the disk = stays

admin habit: ① try it LIVE → ② check it works → ③ write it into the FILE so it stays

/proc /sys /dev /run = see and change the system live (kernel views + RAM) – good for trying first · /etc = the settings programs read at start

B · The test pattern – systemd-run makes a temporary box, -p puts a limit on it, stress-ng makes fake load

```
sudo systemd-run --scope -p <LIMIT> stress-ng <LOAD> --timeout <TIME>
```

`--scope`	`-p <LIMIT>`	`stress-ng <LOAD>`	where does the box go?
box for MY command: output on my screen, deleted when the command ends	MemoryMax = kill · MemoryHigh = slow + swap CPUQuota = throttle · TasksMax = max processes	`--vm 1 --vm-bytes 300M` = 1 worker 300MB `--cpu 1` = busy core · `--fork 4` = forks	with sudo → the system systemd → system.slice (the default place)

C · My test on app01: MemoryMax=100M, no swap, stress-ng wants 300M → killed inside its box only

1 systemd-run	2 stress-ng worker	3 hits the wall	4 OOM killer	5 parent survives	6 box deleted
temporary box `run-ra9665….scope`	wants ~340 MB `total-vm 349924kB`	real RAM ~96 MB `anon-rss 98792kB`	kills the worker 7× · new PID each time	PID 2687 "successful run completed"	command ended nothing left behind

proof in the journal: constraint=CONSTRAINT_MEMCG · oom_memcg=/system.slice/run-ra9665….scope

CONSTRAINT_MEMCG = killed because of the BOX limit, not because app01 ran out of RAM · only that box was touched – services next to it kept running
same idea as a Kubernetes pod "OOMKilled" or Docker --memory

D · Useful checks

`pgrep -a stress-ng` – is a test still running?	`systemctl list-units --all | grep mytest` – old named test boxes?
`sudo pkill stress-ng · systemctl stop mytest` – stop leftovers	`sudo systemctl reset-failed` – clean old failed boxes
`sudo journalctl -k --since "5 min ago" | grep -i killed` – OOM kills in the kernel log	`findmnt · free -h · cat /etc/fstab` – mounts · RAM/swap · mount recipe

Final paragraph – the whole day in one place

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (vmlinuz with its helper initrd.img) is copied from /boot into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in /etc – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, /run in RAM is gone at reboot, /proc /sys /dev are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and WHERE I write it decides if it stays: a live change (sysctl -w, systemd-run, --runtime, mount, ip addr) is a test that is gone at reboot; a file on the disk – usually in /etc – is read at every boot and stays. My test proved it: systemd-run made a temporary box in system.slice with MemoryMax=100M, the OOM killer stopped stress-ng at about 96 MB inside that box only (CONSTRAINT_MEMCG), and the box disappeared when the test ended.

Page 2 of the overview – live test vs permanent and the OOM test

4. Key understanding (in my own words)

Everything is stored on the disk, and RAM starts empty. At power-on the kernel (`vmlinuz` with its helper `initrd.img`) is copied from `/boot` into RAM and becomes the manager; it starts systemd as PID 1. systemd reads its recipes in `/etc` – each read puts a copy of that one file in RAM (page cache) while the original stays on the disk – and from them builds the live things in RAM: the mount table, the services and the cgroup boxes. Every program asks the kernel for files; the kernel gives a copy from RAM, or reads the disk first. Each folder is mounted to a source: disk folders stay, `/run` in RAM is gone at reboot, `/proc`, `/sys` and `/dev` are live kernel information, the NFS share stays on stor01. The work always happens in RAM, and **where I write it decides if it stays**: a live change (`sysctl -w`, `systemd-run`, `--runtime`, `mount`, `ip addr`) is a test that is gone at reboot; a file on the disk – usually in `/etc` – is read at every boot and stays. My test proved it: `systemd-run` made a temporary box in system.slice with `MemoryMax=100M`, the OOM killer stopped stress-ng at about 96 MB inside that box only (`CONSTRAINT_MEMCG`), and the box disappeared when the test ended.

5. Next – the rest of the cgroups lab

1. ~~Step 1 (app01): the cgroup tree.~~ ✅ (Day 10)
2. ~~Step 2 (app01): memory limit → OOM killer.~~ ✅

3. **Soft limit (app01):** `sudo systemd-run --scope -p MemoryHigh=100M stress-ng --vm 1 --vm-bytes 200M --timeout 20s` → slowed and swapped, not killed (`free -h` before/during).
4. **Step 3 (app01):** CPU limit – `CPUQuota=20%` → `top` and `systemd-cgtop` show ~20 %.
5. **Step 4 (stor01):** permanent limits on `archive-worker.service`.
6. Then: first incident drill · L3 Problem in GLPI · snapshot `50-processes`.